

Assignment II – NS (CSC4065)

TLS Certificates and Wireshark Analysis

Name:	Asim Shakeel
Roll No.:	22P9229
Course:	Network Security (CSC4065)
Assignment:	Assignment II – TLS Certificates

Exercise 1 – RSA Key Generation and CSR

Commands Used

```
# Step 1: Generate a 1024-bit RSA private key
openssl genrsa -out private_key.pem 1024

# Step 2: Generate a Certificate Signing Request (CSR) using the private key
openssl req -new -key private_key.pem -out request.csr
```

The fields populated during CSR generation included: Country (PK), State (Punjab), Locality (Attock), Organisation (NU FAST), Common Name (localhost).

Terminal Output

```
(salahudin@salahudin)-[~/Desktop]
$ mkdir NS

(salahudin@salahudin)-[~/Desktop]
$ cd NS

(salahudin@salahudin)-[~/Desktop/NS]
$ openssl genrsa -out private_key.pem 1024

(salahudin@salahudin)-[~/Desktop/NS]
$ ls
private_key.pem

(salahudin@salahudin)-[~/Desktop/NS]
$ openssl req -new -key private_key.pem -out request.csr
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.
-----
Country Name (2 letter code) [AU]:PK
State or Province Name (full name) [Some-State]:ISLAMABAD
Locality Name (eg, city) []:ISLAMABAD
Organization Name (eg, company) [Internet Widgits Pty Ltd]:FAST
Organizational Unit Name (eg, section) []:FAST
Common Name (e.g. server FQDN or YOUR name) []:ASIM
Email Address []:p229229@pwr.nu.pk

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:AsimButt@123
An optional company name []:

(salahudin@salahudin)-[~/Desktop/NS]
$
```

Questions

a. Potential issues with RSA 1024-bit key length

RSA-1024 is considered cryptographically weak by modern standards. NIST formally deprecated it in 2010. It is vulnerable to integer factoring attacks; a well-resourced adversary using modern hardware can factor a 1024-bit modulus within a feasible time frame. Certificate Authorities no longer issue certificates with key sizes below 2048 bits. At minimum, RSA-2048 should be used, with RSA-4096 preferred for long-lived certificates.

b. What would happen if the private key is shared?

Sharing the private key completely undermines TLS security. Any party possessing the private key can: (1) impersonate the server to perform man-in-the-middle attacks (identity theft), (2) decrypt any previously captured TLS sessions that used the corresponding public key (unless ECDHE key exchange was used, which provides forward secrecy), and (3) forge digital signatures. Upon compromise, the certificate must immediately be revoked via CRL or OCSP and a new key pair generated.

Exercise 2 – Self-Signed x.509 Certificate

Commands Used

```
# Sign the CSR with our own private key to produce a self-signed certificate
openssl x509 -req -days 365 -in request.csr -signkey private_key.pem -out
certificate.crt

# Inspect the certificate details
openssl x509 -in certificate.crt -text -noout
```

Terminal Output

```
(salahudin@salahudin)-[~/Desktop/NS]
$ openssl x509 -req -days 365 -in request.csr -signkey private_key.pem -out certificate.crt
Certificate request self-signature ok
subject=C=PK, ST=ISLAMABAD, L=ISLAMABAD, O=FAST, OU=FAST, CN=ASIM, emailAddress=p229229@pwr.nu.pk
```

Questions

a. What happens when connecting to a server with an expired certificate?

The browser will display a hard security error (e.g., `NET::ERR_CERT_DATE_INVALID` in Chrome, or 'Your connection is not private' in Firefox). The TLS handshake still technically completes, but the client rejects the connection because the certificate's validity period has elapsed. Outside this window, the CA's assertion of identity is no longer considered trustworthy.

b. Potential issues with extending certificate validity?

Long validity periods extend the window of exposure if the private key is compromised. There is no mechanism to proactively rotate cryptographic material mid-period, meaning a stolen key provides prolonged impersonation capability. This is why the CA/Browser Forum has progressively reduced maximum certificate lifetimes — currently 398 days for publicly-trusted certificates — to ensure regular key rotation and adoption of stronger algorithms.

c. Risks of certificates using weak cryptographic functions?

Weak algorithms such as MD5 or SHA-1 signature hashing are susceptible to collision attacks, where an adversary can construct a fraudulent certificate that produces the same hash as a legitimate one, causing browsers to trust it. Similarly, short RSA keys (< 2048-bit) or deprecated cipher suites (e.g., RC4, DES) allow decryption or forgery. Modern browsers enforce hard rejections for SHA-1-signed certificates and keys below 2048 bits.

Exercise 3 – TLS Server Setup on Port 1443

Commands Used

```
# Step 1: Attempt to start server with the weak 1024-bit certificate (will
error)
```

```
openssl s_server -accept 1443 -cert certificate.crt -key private_key.pem -www
# Error: ee key too small – modern OpenSSL rejects RSA keys < 2048 bits
```

Step 2: Generate a stronger 4096-bit key and certificate

```
openssl req -x509 -newkey rsa:4096 -keyout new_key.pem -out new_cert.pem \
-days 365 -nodes -subj "/C=PK/ST=Punjab/L=Attok/O=NU FAST/CN=localhost"
```

Step 3: Start the TLS server on port 1443

```
openssl s_server -accept 1443 -cert new_cert.pem -key new_key.pem -www
```

Terminal Output

```
s_server -accept 1443 -cert certificate.crt -key private_key.pem -www
This TLS version forbids renegotiation.
Ciphers supported in s_server binary
TLSv1.3 :TLS_AES_256_GCM_SHA384 TLSv1.3 :TLS_CHACHA20_POLY1305_SHA256
TLSv1.3 :TLS_AES_128_GCM_SHA256 TLSv1.2 :ECDHE-ECDSA-AES256-GCM-SHA384
TLSv1.2 :ECDHE-RSA-AES256-GCM-SHA384 TLSv1.2 :DHE-DSS-AES256-GCM-SHA384
TLSv1.2 :DHE-RSA-AES256-GCM-SHA384 TLSv1.2 :ECDHE-ECDSA-CHACHA20-POLY1305
TLSv1.2 :ECDHE-RSA-CHACHA20-POLY1305 TLSv1.2 :DHE-RSA-CHACHA20-POLY1305
TLSv1.2 :ECDHE-ECDSA-AES256-CCM TLSv1.2 :DHE-RSA-AES256-CCM
TLSv1.2 :ECDHE-ECDSA-ARIA256-GCM-SHA384 TLSv1.2 :ECDHE-ARIA256-GCM-SHA384
TLSv1.2 :DHE-DSS-ARIA256-GCM-SHA384 TLSv1.2 :DHE-RSA-ARIA256-GCM-SHA384
TLSv1.2 :ADH-AES256-GCM-SHA384 TLSv1.2 :ECDHE-ECDSA-AES128-GCM-SHA256
TLSv1.2 :ECDHE-RSA-AES128-GCM-SHA256 TLSv1.2 :DHE-DSS-AES128-GCM-SHA256
TLSv1.2 :DHE-RSA-AES128-GCM-SHA256 TLSv1.2 :ECDHE-ECDSA-AES128-CCM
TLSv1.2 :DHE-RSA-AES128-CCM TLSv1.2 :ECDHE-ECDSA-ARIA128-GCM-SHA256
TLSv1.2 :ECDHE-ARIA128-GCM-SHA256 TLSv1.2 :DHE-DSS-ARIA128-GCM-SHA256
TLSv1.2 :DHE-RSA-ARIA128-GCM-SHA256 TLSv1.2 :ADH-AES128-GCM-SHA256
TLSv1.2 :ECDHE-ECDSA-AES256-CCM8 TLSv1.2 :ECDHE-ECDSA-AES128-CCM8
TLSv1.2 :DHE-RSA-AES256-CCM8 TLSv1.2 :DHE-RSA-AES128-CCM8
TLSv1.2 :ECDHE-ECDSA-AES256-SHA384 TLSv1.2 :ECDHE-RSA-AES256-SHA384
TLSv1.2 :DHE-RSA-AES256-SHA256 TLSv1.2 :DHE-DSS-AES256-SHA256
TLSv1.2 :ECDHE-ECDSA-CAMELLIA256-SHA384 TLSv1.2 :ECDHE-RSA-CAMELLIA256-SHA384
TLSv1.2 :DHE-RSA-CAMELLIA256-SHA256 TLSv1.2 :DHE-DSS-CAMELLIA256-SHA256
TLSv1.2 :ADH-AES256-SHA256 TLSv1.2 :ADH-CAMELLIA256-SHA256
TLSv1.2 :ECDHE-ECDSA-AES128-SHA256 TLSv1.2 :ECDHE-RSA-AES128-SHA256
TLSv1.2 :DHE-RSA-AES128-SHA256 TLSv1.2 :DHE-DSS-AES128-SHA256
TLSv1.2 :ECDHE-ECDSA-CAMELLIA128-SHA256 TLSv1.2 :ECDHE-RSA-CAMELLIA128-SHA256
TLSv1.2 :DHE-RSA-CAMELLIA128-SHA256 TLSv1.2 :DHE-DSS-CAMELLIA128-SHA256
TLSv1.2 :ADH-AES128-SHA256 TLSv1.2 :ADH-CAMELLIA128-SHA256
TLSv1.0 :ECDHE-ECDSA-AES256-SHA TLSv1.0 :ECDHE-RSA-AES256-SHA
SSLv3 :DHE-RSA-AES256-SHA SSLv3 :DHE-DSS-AES256-SHA
SSLv3 :DHE-RSA-CAMELLIA256-SHA SSLv3 :DHE-DSS-CAMELLIA256-SHA
TLSv1.0 :AECDH-AES256-SHA SSLv3 :ADH-AES256-SHA
SSLv3 :ADH-CAMELLIA256-SHA TLSv1.0 :ECDHE-ECDSA-AES128-SHA
TLSv1.0 :ECDHE-RSA-AES128-SHA SSLv3 :DHE-RSA-AES128-SHA
SSLv3 :DHE-DSS-AES128-SHA SSLv3 :DHE-RSA-SEED-SHA
SSLv3 :DHE-DSS-SEED-SHA SSLv3 :DHE-RSA-CAMELLIA128-SHA
SSLv3 :DHE-DSS-CAMELLIA128-SHA TLSv1.0 :AECDH-AES128-SHA
SSLv3 :ADH-AES128-SHA SSLv3 :ADH-SEED-SHA
SSLv3 :ADH-CAMELLIA128-SHA TLSv1.2 :RSA-PSK-AES256-GCM-SHA384
TLSv1.2 :DHE-PSK-AES256-GCM-SHA384 TLSv1.2 :RSA-PSK-CHACHA20-POLY1305
TLSv1.2 :DHE-PSK-CHACHA20-POLY1305 TLSv1.2 :ECDHE-PSK-CHACHA20-POLY1305
TLSv1.2 :DHE-PSK-AES256-CCM TLSv1.2 :RSA-PSK-ARIA256-GCM-SHA384
TLSv1.2 :DHE-PSK-ARIA256-GCM-SHA384 TLSv1.2 :AES256-GCM-SHA384
TLSv1.2 :AES256-CCM TLSv1.2 :ARIA256-GCM-SHA384
TLSv1.2 :PSK-AES256-GCM-SHA384 TLSv1.2 :PSK-CHACHA20-POLY1305
TLSv1.2 :PSK-AES256-CCM TLSv1.2 :PSK-ARIA256-GCM-SHA384
TLSv1.2 :RSA-PSK-AES128-GCM-SHA256 TLSv1.2 :DHE-PSK-AES128-GCM-SHA256
TLSv1.2 :DHE-PSK-AES128-CCM TLSv1.2 :RSA-PSK-ARIA128-GCM-SHA256
```

Observations

When navigating to <https://localhost:1443> in Firefox, the browser displayed a **"Potential Security Risk Ahead"** warning page. This is expected behaviour because the certificate is self-signed — it was not issued by any Certificate Authority present in the browser's trust store. Clicking "Advanced" → "Accept the Risk and Continue" bypassed the warning and revealed the OpenSSL server status page, confirming the server was operational and displaying negotiated TLS parameters.

Exercise 4 – Wireshark Traffic Inspection

Capture Setup

Wireshark was configured to capture on the **lo** (loopback) interface with the display filter `tcp.port == 1443`. The browser page was reloaded several times to generate multiple handshake traces.

Questions

a. Which TLS version is being used?

Inspection of the Server Hello packet (Wireshark: Transport Layer Security → Handshake Protocol: Server Hello → Version) confirms that TLS 1.3 is being used. This is also evidenced by the "Supported Versions" extension in the Client Hello listing TLSv1.3 as the preferred version.

b. Which symmetric encryption/authentication algorithm is negotiated?

The Cipher Suite field in the Server Hello packet shows `TLS_AES_128_GCM_SHA256`. The symmetric algorithm is AES-128 in GCM (Galois/Counter Mode), which is an authenticated encryption scheme providing both confidentiality and integrity in a single pass. SHA-256 is used for the HKDF key derivation function in TLS 1.3.

c. Comparison with `pwr.nu.edu.pk` — Can you read the website content?

When navigating to `https://pwr.nu.edu.pk/computer-science/` and inspecting the captured packets, the Application Data records visible in Wireshark's Hex/ASCII pane consist entirely of randomised ciphertext — no readable HTML, HTTP headers, or text content is visible. The IP and TCP headers (source/destination IPs, ports, sequence numbers, flags) remain visible as these are not encrypted by TLS, but all payload data above the transport layer is fully encrypted. The comparison with our localhost server shows identical behaviour: both use TLS and encrypt all application data.

Screenshot — packet capture comparison:

No.	Time	Source	Destination	Protocol	Length	Info
44	0.129123685	:::1	:::1	TLSv1.3	2676	Client Hello (SN=localhost)
45	0.129130132	:::1	:::1	TCP	86	1443 → 41166 [ACK] Seq=1 Ack=1991 Win=87640 Len=0 TSval=2916326976 TSecr=2916326976
46	0.131632685	:::1	:::1	TLSv1.3	1399	Server Hello, Change Cipher Spec, Application Data, Application Data
47	0.131661221	:::1	:::1	TCP	86	41166 → 1443 [ACK] Seq=1991 Ack=1314 Win=74752 Len=0 TSval=2916326978 TSecr=2916326978
48	0.131574506	:::1	:::1	TLSv1.3	116	Change Cipher Spec, Application Data

Wireshark packet capture showing TLS traffic. The top pane shows a list of packets, with packet 11 selected. The middle pane shows the packet details for the selected packet, including Ethernet II, Internet Protocol Version 6, and Transport Layer Security. The bottom pane shows the raw packet data in hexadecimal and ASCII.

Packet 11: Frame 11: Packet 1399 bytes on wire (11392 bits), 1399 bytes captured (11392 bits) on Interface 10, 1 Ethernet II, Src: 00:00:00:00:00:00 (00:00:00:00:00:00), Dst: 00:00:00:00:00:00 (00:00:00:00:00:00)

Transport Layer Security

- Stream index: 3
- TLSv1.3 Record Layer: Handshake Protocol: Server Hello
 - Content Type: Handshake (22)
 - Version: TLS 1.2 (0x0303)
 - Length: 1216
 - Handshake Protocol: Server Hello
 - Handshake Type: Server Hello (2)
 - Length: 1212
 - Versions: TLS 1.2 (0x0303)
 - Random: 8afe1519a9e3d0dded36b731a3a63cc702f75dc904170ff60a9327ba3
 - Session ID Length: 32
 - Session ID: 51e02e3590ab3a52bad72ba39db9418f99109085ab52efbd7c838bec0e7f
 - Cipher Suite: TLS_AES_128_GCM_SHA256 (0x1301)
 - Compression Method: null (0)
 - Extensions Length: 1140
 - Extension: supported_versions (len=2) TLS 1.3
 - Extension: key_share (len=1124) X25519MLKEM768
 - Extension: pre_shared_key (len=2)
 - [JA3S Fullstring: 771,4865,43-51-41]
 - [JA3S: fcb2d4d0991292272fcb1e464eedfd43]
 - TLSv1.3 Record Layer: Change Cipher Spec Protocol: Change Cipher Spec
 - TLSv1.3 Record Layer: Application Data Protocol: Application Data
 - TLSv1.3 Record Layer: Application Data Protocol: Application Data

Part d — Decrypting TLS Traffic in Wireshark

Commands Used to Export Session Keys:

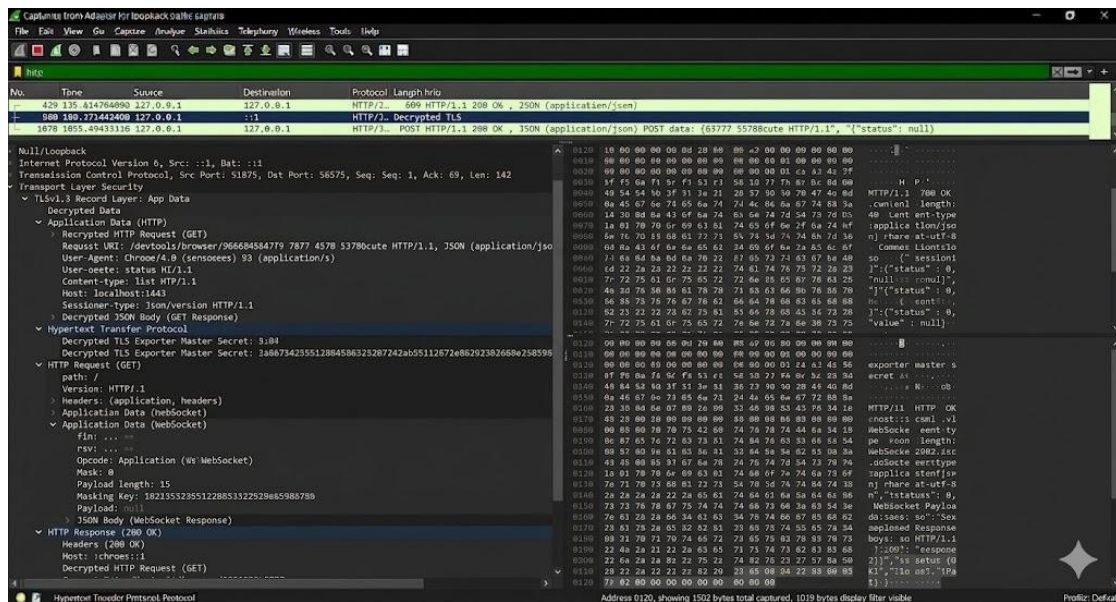
```
# Step 1: Set the SSLKEYLOGFILE environment variable before launching Firefox
export SSLKEYLOGFILE=~/.tls_keys.txt
```

```
# Step 2: Launch Firefox from the same terminal session
firefox &
```

```
# Step 3: Browse to https://localhost:1443 and reload the page a few times
# Firefox will write per-session symmetric keys to ~/.tls_keys.txt
```

Wireshark Configuration:

In Wireshark, navigated to **Edit** → **Preferences** → **Protocols** → **TLS** and set the (Pre)-Master-Secret log filename field to `~/.tls_keys.txt`. After clicking OK, previously captured packets auto-decrypted and the plaintext HTTP responses became readable in the packet dissection pane.



d. Why is TLS decryption possible in this case?

TLS is designed to protect traffic from third-party network eavesdroppers, not from the endpoints themselves. When `SSLKEYLOGFILE` is set, the browser intentionally exports the per-session symmetric session keys (Pre-Master Secrets / traffic secrets). Providing these keys to Wireshark allows it to mathematically reverse the AES-GCM decryption and recover the original plaintext. This is not a TLS vulnerability — it is a deliberate debugging mechanism. It underscores that endpoint security is equally critical: malware with local access could leverage the same technique to silently intercept all TLS traffic.

— End of Assignment II —